Passing Pointers to Functions

Understanding how to pass pointers to functions in C programming

Methods of Passing Arguments

C programming allows in FUNCTIONS that arguments can generally be passed to functions in one of the two following ways:

Pass by Value

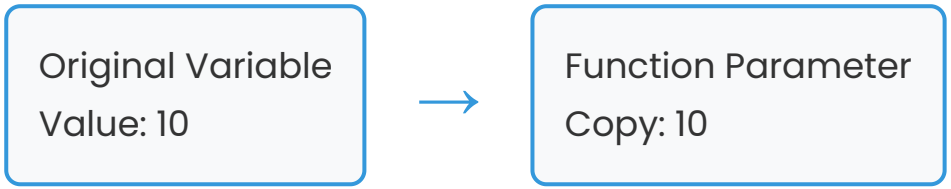
A copy of values is passed to the function

Pass by Reference

Address of variables is passed to the function

Pass by Value Method

In first method, when arguments are passed by value, a copy of values of actual arguments is passed to calling function. Thus, any changes made to variables inside function will have no effect on variables used in actual argument list.



When the function modifies the parameter, only the local copy is changed, not the original variable.

Pass by Reference Method

However, when arguments are passed by reference (i.e. when a pointer is passed as an argument to a function), address of a variable is passed. The contents of that address can be accessed freely, either in the called or calling function. Therefore, function called by reference can change value of variable used in call.

When the function modifies the value at the address, the original variable is changed because both the function and caller are accessing the same memory location.

Example: Swap Functions

Write a program to swap values using pass by value and pass by reference methods.

Program Demonstrating Both Methods

```
/* Program that illustrates the difference between ordinary arguments, which are passed by value, and pointer
arguments, which are passed by reference */
# include<stdio.h>
main()
{
  int x = 10;
  int y = 20;
  void swapVal ( int, int ); /* function prototype */
  void swapRef (int*, int*); /*function prototype*/
  printf("PASS BY VALUE METHOD\n");
  printf ("Before calling function swapVal x=%d y=%d",x,y);
  swapVal (x, y); /* copy of arguments are passed */
  printf("\n After calling function swapVal x=%d y=%d",x,y);
  printf("\n\n PASS BY REFERENCE METHOD");
  printf ("\n Before calling function swapRef x=%d y=%d",x,y);
  swapRef (&x,&y); /*address of arguments are passed */
```

```
  printf("\nAfter calling function swapRef x=%d y=%d",x,y);
}
```


SwapVal Function (Pass by Value)

Function Using Pass by Value

```
/* Function using pass by value method */
void swapVal (int x, int y)
{
  int temp;
   temp = x;
   x = y;
   y = temp;
  printf ("\nWithin function swapVal x=%d y=%d",x,y);
  return;
}
```

Key Points

Local Variables

x and y are local copies of the original values

Swap Operation

Values are swapped locally but original variables remain unchanged

SwapRef Function (Pass by Reference)

Function Using Pass by Reference

```c
/*Function using pass by reference method*/
void swapRef (int *px, int *py)
{
  int temp;
  temp = *px;
  *px = *py;
  *py = temp;
  printf ("\nWithin function swapRef *px=%d *py=%d",*px,*py);
  return;
}
```

Key Points

Pointer Parameters

px and py are pointers to the original variables

Swap Operation

Values at the addresses are swapped, changing the original variables

Output of Swap Example

Program Output

```
PASS BY VALUE METHOD
Before calling function swapVal x=10 y=20
Within function swapVal x=20 y=10
After calling function swapVal x=10 y=20

PASS BY REFERENCE METHOD

Before calling function swapRef x=10 y=20
Within function swapRef *px=20 *py=10
After calling function swapRef x=20 y=10
```

Explanation

Pass by Value

In function swapVal, arguments x and y are passed by value. So, any changes to arguments are local to the function in which changes occur. Note values of x and y

remain unchanged even after exchanging values of x and y inside the function swapVal.

Pass by Reference

In function swapRef, addresses of x and y are passed, allowing the function to modify the original values.

Function Returning More Than One Value

Using call by reference method we can make a function return more than one value at a time, which is not possible in call by value method. The following program will make you the concept very clear.

Single Return
Only one value

→

Multiple Returns
Through pointers

Example: Perimeter and Area

Write a program to find the perimeter and area of a rectangle, if length and breadth are given by the user.

Program to Find Perimeter and Area

```c
/* Program to find perimeter and area of a rectangle*/
#include<stdio.h>
void main()
{
  float len,br;
  float peri, ar;
  void periarea(float length, float breadth, float *, float *);
  printf("\nEnter length and breadth of a rectangle in metres: \n");
  scanf("%f %f",&len,&br);
  periarea(len,br,&peri,&ar);
  printf("\nPerimeter of rectangle is %f metres", peri);
  printf("\nArea of rectangle is %f sq. metres", ar);
}
void periarea(float length, float breadth, float *perimeter, float *area)
{
  *perimeter = 2 * (length +breadth);
  *area = length * breadth;
}
```


Output of Perimeter and Area

Program Output

```
Enter length and breadth of a rectangle in metres:
23.0 3.0
Perimeter of rectangle is 52.000000 metres
Area of rectangle is 69.000000 sq. metres
```

Explanation

Perimeter Calculation

2 × (length + breadth)

Area Calculation

length × breadth

Pointer Parameters

Both values returned through pointers to the calling function

Function Returning a Pointer

A function can also return a pointer to calling program, way it returns an int, a float or any other data type. To return a pointer, a function must explicitly mention in the calling program as well as in the function prototype.

📝 Function Declaration

```
int *myFunction()
```

To declare a function returning a pointer as shown above.

Important Point

Second point to remember is that, it is to return the address of a local variable outside the function, so you would have to define the local variable as static variable.

Example: Returning a Pointer

Now, consider the following function which will generate 10 random numbers and return them using an array name which represents a pointer, i.e., address of first array element.

Random Number Generator Function

```c
#include<stdio.h>
#include<time.h>
/* function to generate and return random numbers. */
int *getRandom( )
{
  static int r[10];
  int i;
  /* set the seed */
  srand((unsigned)time(NULL));
  for ( i = 0; i < 10; ++i)
  {
    r[i] = rand();
    printf("%d\n", r[i] );
  }
```

```
  return r;
}
```


Main Function Calling getRandom

Main Function to Call getRandom

```
/* main function to call above defined function */
int main ()
{
  /* a pointer to an int */
  int *p;
  int i;
  p = getRandom();
  for ( i = 0; i < 10; i++ )
  {
    printf("*(p + [%d]) : %d\n", i, *(p + i) );
  }
```

```
  return 0;
}
```


Output of Random Number Example

Program Output

```
1523198053
1187214107
1108300978
430494959
1421301276
930971084
123250484
106932140
1604461820
149169022

*(p + [0]) : 1523198053
*(p + [1]) : 1187214107
*(p + [2]) : 1108300978
```

```
*(p + [3]) : 430494959
*(p + [4]) : 1421301276
*(p + [5]) : 930971084
*(p + [6]) : 123250484
*(p + [7]) : 106932140
*(p + [8]) : 1604461820
*(p + [9]) : 149169022
```

Explanation

Static Array

The array r is declared as static so it persists after the function returns

Pointer Return

The function returns the address of the first element of the array

Pointer Arithmetic

p + i accesses the i-th element of the array

Passing pointers to functions provides powerful capabilities in C programming, allowing

functions to modify original variables and return multiple values through pointers.